BUBBLE

Social Media Platform To Connect with What

Introduction

Introduction

Digital communication and community engagement are fundamental aspects of modern social interaction, yet conventional social platforms have become increasingly fragmented, noisy, and impersonal. Users are often forced to juggle disjointed messaging tools, overwhelming public feeds, and uncurated group forums just to maintain authentic relationships and shared-interest communities. This lack of focused organization leads to digital fatigue, context switching, and communication friction for both active community creators and everyday participants.

Beyond organizational clutter, mainstream social ecosystems face severe challenges regarding privacy, data governance, and trust. The unchecked proliferation of unverified profiles, intrusive algorithmic feeds, and insecure data handling degrades confidence in personal digital interactions. Furthermore, relying on fragmented third-party channels to exchange media, coordinate discussions, and store personal content introduces critical vulnerabilities surrounding data ownership, session security, and network scalability.

To address these systemic challenges, this project proposes the development of Bubble, a secure, fluid mobile-based community and messaging platform designed to centralize social spaces into intuitive, dedicated Bubbles. The application empowers users to seamlessly discover and engage within interest-based communities, curate personalized content feeds, and communicate in real time through secure, authenticated messaging channels. Supported by robust role-based moderation and cloud-native architecture, Bubble establishes a private, scalable, and verified ecosystem for modern digital connections

Problems

Statement OF The problem

2.1 Lack of a Centralized Platform

People have to use different apps for group posts, messaging, and community updates. Switching between apps makes it hard to manage everything in one place.

2.2 Cluttered Feeds and Post Organization

Social feeds are full of random ads and suggested posts. This makes it hard for users to see posts from the communities and friends they actually care about.

2.3 Inefficient Community and User Search

Users have limited ways to search for specific groups or friends by topic or interest. Finding the right community is slow and frustrating.

2.4 Disconnected Group Feeds and Private Chat

Feeds and direct messages are kept in separate areas. When users want to talk to someone about a post, they have to leave the feed and start a separate chat.

2.5 Limited Privacy and Circle Controls

Most apps only let accounts be fully public or fully private. Users cannot easily share posts with just a small group or close friends.

2.6 Weak Moderation and Group Management

Group owners do not have simple tools to delete spam, report bad posts, or remove rule-breaking users. This makes communities messy and unsafe

Objective

General Objective

The general objective of this project is to create Bubble, a secure and easy-to-use mobile platform that replaces fragmented social tools by providing dedicated community circles, relevant content feeds, direct chat features, and safe group management in one centralized system

Specific Objective

To design a simple mobile interface for easy navigation between feeds, search, and chats.

To implement secure user authentication for registration, login, and account protection.

To allow users to create and customize "Bubbles" as public or private community groups.

To enable post creation and media sharing so users can publish updates and photos easily.

To build an organized community feed that displays relevant posts without ads or clutter.

To develop a search feature that helps users quickly find groups and friends by keyword or topic.

To provide flexible privacy controls for managing profile visibility and group join permissions.

To build community moderation tools for admins to delete spam and remove rule-breaking members.

To create an in-app notification system to alert users about new messages, comments, and invites.

To implement real-time messaging for instant one-on-one and group chats.

Functions

Functions of a Regular User



Create And Update Account and Profile



Search for communities (Bubbles), posts, and other users using keywords.



Join public Bubbles or send join requests to private Bubbles.



Create new posts with text and photos inside joined Bubbles.



React, comment on, and interact with posts shared by others.



Edit or delete their own posts and comments at any time.



Send and receive real-time direct messages with friends.



Send, accept, or decline friend requests.



Report offensive posts, spam, or abusive users.



Receive in-app notifications for new messages, replies, and mentions.



Create a new Bubble, choose a name, write a description, and select public or private status.



Review, accept, or decline member requests for private Bubbles they joined

Functions of Admin



Manage and monitor all registered user accounts across the platform.



Review system-wide reported content, spam, and abusive behavior.



Temporarily suspend or permanently ban accounts that violate platform terms.



Delete harmful communities or illegal content when necessary.



View platform usage stats, such as total users, active Bubbles, and daily posts.

Limitation

Limitation



Does not have external object store



OTP delivery dependability



Used 3rd party icons and UI inconsistency

Architecture

Programing Languages



Golang

Used for the Service Side
logic & businesses



Dart

For front end
development

Used Tools



Postgresql

Database engine



Flutter

For front end development



Mongo

Database for Chat Micro service



Redis

To Connect The chat Micro Service with the main back-end. and for



Firebase

To Push Notification To users Device

Fiber

Used as Web Socket on both servers

Architecture

The Projects follows 3 code bases that have there own Architecture :

Frontend (Flutter):

Feature-First Architecture: Split into modular domain folders (auth, home, chat, etc.).

Riverpod: Handles state management and dependency injection.

Main Backend (Go + Fiber):

Hexagonal Architecture: Domain $\to$ Service $\to$ Repository layers with constructor injection.

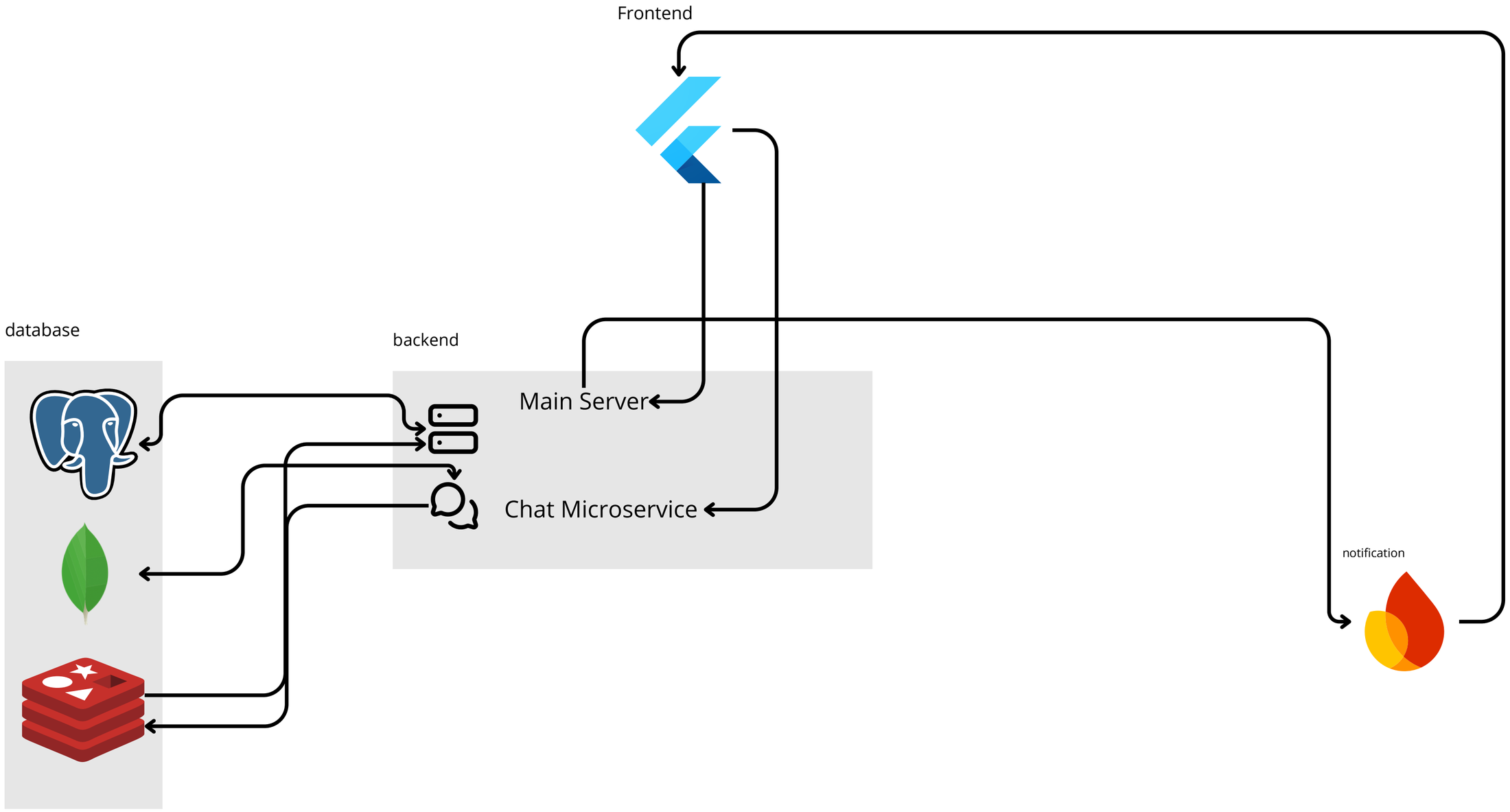
PostgreSQL: Stores relational data

Chat Microservice (Go + Fiber):

Real-time WebSockets: Manages live, bidirectional messaging.

MongoDB: Stores fast, unstructured message and conversation history.

when users are offline.


Frontend
database
backend
Main Server
Chat Microservice
notification



Nahom Ketema

CTC-191-26

THANK YOU